Lab 2: Dynamic Programming

Morgan Cooper
MSDS 684 — Reinforcement Learning

March 20, 2026

Section 1: Project Overview

This project is focused on solving a known Markov Decision Process (MDP) with dynamic programming (DP). DP requires complete knowledge of the environment that the agent will be operating within. This is different than Lab 1 where the agent had no knowledge of the environment and had to store the expected action-reward values in a Q-Table. Since we already know the world view of the environment via the MDP, we can focus on planning and developing an optimal policy.

This project explores concepts from Sutton and Barto (2020) including policy evaluation, policy improvement, policy iteration, and value iteration. A policy is a mapping over each state to an action. It explains which actions to take given each state. A value function provides a cumulative expected value for each state by combining the current reward and all future rewards. Policy evaluation involves identifying a value function for a given policy. Policy improvement focuses on starting with a value function and improving the policy. Policy evaluation asks "what is the value of each state under this policy?" and policy improvement asks "given the value of each state, can we pick better actions?"

Policy iteration works by running policy evaluation loops over each state and collecting their values. During each loop, the reward continues to propagate up the chain of states until the value of each state stops changing, and the value function converges. Then a loop is run performing policy improvement where the policy is updated by selecting the action with the highest value at each state. This process of evaluation and policy improvement is run until there is no more change and the optimal policy is found.

Value iteration is a different algorithm that runs loops over each state, collecting the best value at each state by checking all possible actions. This repeats until the value function stabilizes, then a final loop is run to collect the optimal policy by selecting the action with the highest value for each state.

I expect that all four DP algorithms will find the optimal solution, but I anticipate in-place value iteration to work quicker than the rest. I also think the stochastic environments will be more difficult to find a solution for since they incorporate randomness into the action outcomes. The agent will be required to think differently relative to the stochastic

environment.

To work through these different DP algorithms, this project utilizes two environments:

Custom GridWorld:

- State space: 16 discrete states (4×4 grid), integer-encoded
- Action space: 4 discrete (up, right, down, left)
- Rewards: -1 per step, 0 at goal — incentivizes shortest paths
- Terminal condition: reaching the goal state
- Two configurations: deterministic (100% intended) and stochastic (80/10/10)
- Obstacles that block movement

FrozenLake-v1:

- State space: 16 discrete states (4×4 grid), from Gymnasium
- Action space: 4 discrete (left, down, right, up)
- Rewards: 0 for each step, +1 for reaching the goal
- Terminal conditions: reaching the goal or falling into a hole
- Stochastic by default ($\frac{1}{3}$ probability for each direction)

Section 2: Deliverables

GitHub Repository

`GitHub Repository: https://github.com/cooper-rm/rl-dynamic-programming`

Implementation Summary

For this project I implemented a custom GridWorld environment following Gymnasium's API with a full transition model P[s][a], specifically to facilitate DP. For GridWorld, I implemented four DP algorithms: synchronous policy iteration, synchronous value iteration, in-place policy iteration, and in-place value iteration. I ran experiments in both deterministic and stochastic environments, and applied all the same ideas to Gymnasium's FrozenLake-v1. The key hyperparameters were gamma=0.99, theta=1e-8, and a step reward of -1.

Key Results & Analysis

In the deterministic environment, all four algorithms converged successfully to the same optimal policy (Figure 1). Since the step reward was -1, states near the goal have values close to 0, whereas states that are far away from the goal are more negative.

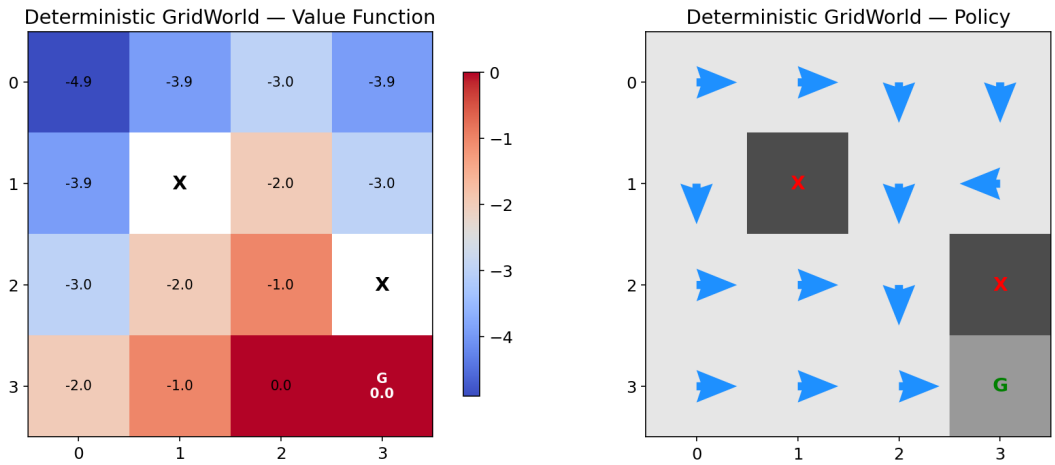

Figure 1: Deterministic GridWorld value function (left) and optimal policy (right), computed via VI In-Place. The heatmap displays $V(s)$ at each grid cell with obstacles marked as X and the goal marked as G. The quiver plot shows the greedy action at each non-terminal state.

In the stochastic environment (Figure 2), the values became even more negative everywhere because each action no longer leads to a guaranteed state. The policy also shifted towards avoiding risky positions where a slip would cost the agent more penalty.

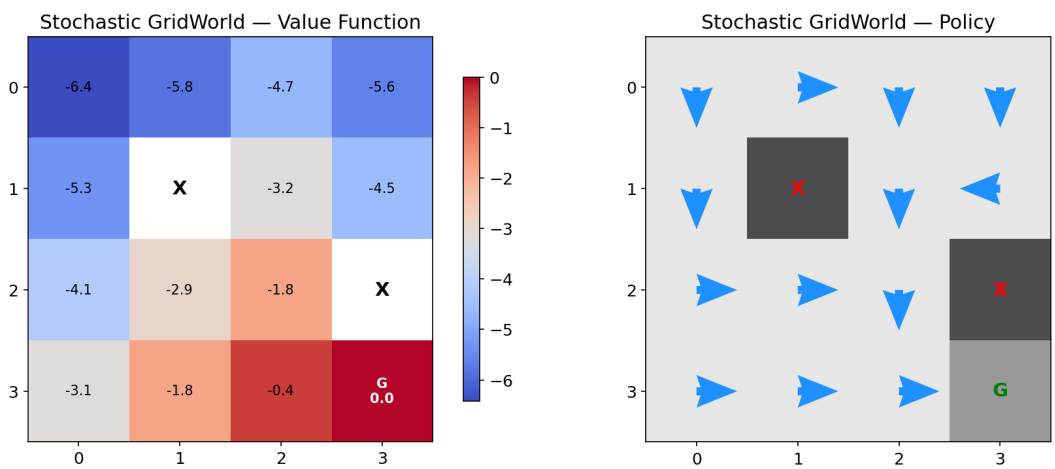

Figure 2: Stochastic GridWorld value function (left) and optimal policy (right), computed via VI In-Place with slip_prob = 0.1. Same layout as Figure 1 but with 80/10/10 transition probabilities. The heatmap shows $V(s)$ and the quiver plot shows the greedy action at each state.

As shown in Figure 3, in-place algorithms converge quicker than synchronous algorithms. This is because values propagate faster through all of the states in the environment. Faster propagation is a result of in-place updates having access to state $N-1$'s new value in the

same sweep, whereas synchronous algorithms must wait for the entire sweep to finish before seeing state value changes.

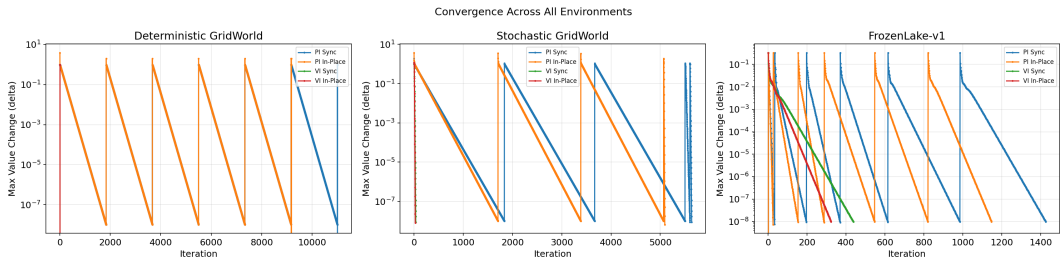

Figure 3: Convergence curves showing max value change (delta) on a log scale per sweep across all three environments. Each line represents one of the four algorithm variants (PI Sync, PI In-Place, VI Sync, VI In-Place).

Finally, in the FrozenLake-v1 (Figure 4) application we see the exact same reaction to determinism, stochasticity, in-place, and synchronous algorithms. Despite the differing environment structure, the same algorithms converge the quickest to an optimal policy. This validates my hypothesis that all four algorithms converge to the same optimal policy regardless of environment structure, and that the in-place value iteration algorithm converges the fastest.

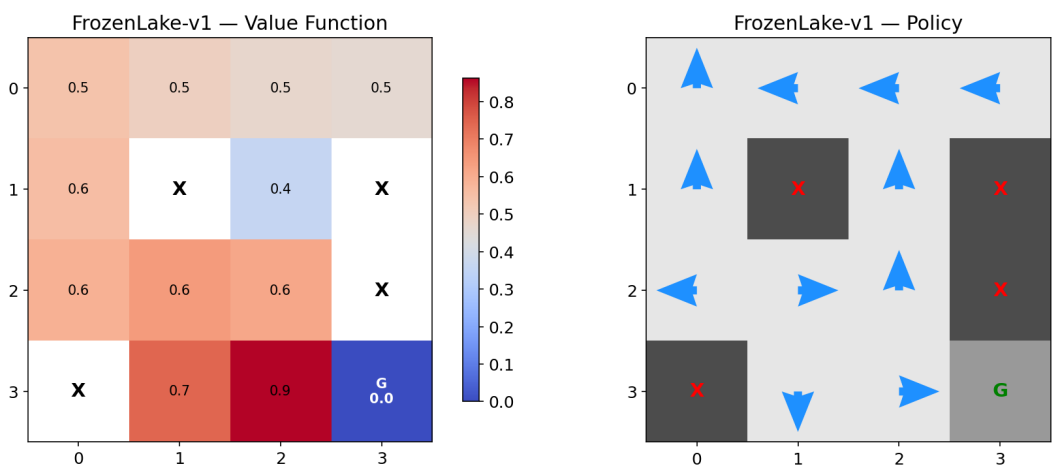

Figure 4: FrozenLake-v1 value function (left) and optimal policy (right), computed via VI In-Place. Holes are marked as X (terminal, reward = 0) and the goal as G (terminal, reward = 1). The environment is stochastic with $\frac{1}{3}$ probability for each direction.

Section 3: AI Use Reflection

Initial Interaction

Initially, I asked Claude Code to review the materials for this weeks Lab and create a notebook using markdown only first to outline the exact steps that need to be completed.

Then I stepped through each section one at a time asking Claude to explain the concepts and what I need to understand before implementing any code. This allowed me to converese about the ideas in this lab before executing. Additionally, I asked Claude to document verbatim every thing I say and everything it says throughout the interaction in a conversation file so I could review what we discussed at anytime.

Iteration Cycle

Iteration 1: Confusing state values with action values

After completing the code portion, I did not find that Claude made any large mistakes. However, I spent a large amount of time reviewing each concept as I completed each section of this report. In section one, I reviewed the conversation log and continued to ask questions. Something I got wrong that Claude corrected me on was that policy evaluation computes the value that each action is worth. My early understanding was that each action results in some reward, therefore the value function was in my mind concerned with the reward that each action would produce at each state. While this sounds right, it is incorrect. An agent takes an action, which results in a transition from one state to another. The reward is a result of the transition, but is tracked by state. This is because in stochastic environments, one action may lead to multiple different states. So, given this correct understanding, I changed my statement to be: "Policy evaluation computes the value of each STATE (not action)."

Iteration 2: Understanding why evaluation needs many sweeps

When working through the specific algorithms, I had a hard time grasping the exact differences between policy evaluation, improvement, and iteration, along with value iteration. I understood the general concept but needed to get precise. I worked through the differences piece by piece, along with reviewing the code, until I finally grasped the concepts. After working through simple examples on paper (in the terminal), I finally got the differences of each algorithm. Claude explained, and I verified with statements like "So with policy iteration, we run a ton of loops till convergence and then run an improvement sweep once, repeating these steps until the policy no longer improves."

Iteration 3: Pylance type hint errors

While I did not find any specific logic errors from Claude's implementation, there were linting issues from Pylance. Python's type checker flagged two cases where the code used default values that did not match the declared types. I worked with Claude to update the type annotations so the linter accepted them without changing any logic.

Critical Evaluation

Early on in the discussion of each algorithm, I found Claude to provide vague or overly simplified explanations on each topic. For this reason, I had to go deeper and push to get more specific until I fully understood the material. I do, however, think this was the best way I could have learned and worked through the materials. If I had been given an overly complex example, there is a chance I would have frozen and spent more time than needed to get a correct understanding. I would have likely asked for a much simpler explanation.

After everything clicked, I was able to verify that all four algorithms converged to the same solution and matched the anticipated behavior.

Learning Reflection

The most difficult challenge presented during this lab was clearly understanding the distinction between each algorithm and the difference between states and actions in the value function. Once I specifically worked through each detail of each algorithm, the environment visuals made complete sense. I now feel confident in understanding DP as it relates to MDPs and how in DP the model must be present.

Section 4: Speaker Notes

- **Problem:** PLACEHOLDER
- **Method:** PLACEHOLDER
- **Design choice:** PLACEHOLDER
- **Key result:** PLACEHOLDER
- **Insight:** PLACEHOLDER
- **Challenge:** PLACEHOLDER
- **Connection:** PLACEHOLDER

References

1. Sutton, R. S., & Barto, A. G. (2018). *Reinforcement learning: An introduction* (2nd ed.). MIT Press.
2. Anthropic. (2025). Claude Code [Large language model CLI tool]. `https://claude.ai`
3. PLACEHOLDER: Add any additional references used.